

Django Models, ORM, Forms & Admin

Database models, ORM queries, migrations, HTML/Model Forms, Django Admin permissions, and MySQL configuration.

1. Models & Object-Relational Mapping (ORM)

A **Model** is the single, definitive source of truth about your data. Each model class maps to a single database table, where attributes represent database fields. Django's **ORM** lets developers interact with SQL databases using pure Python code.

```
from django.db import models
class Category(models.Model):
    name = models.CharField(max_length=100)
class MenuItem(models.Model):
    title = models.CharField(max_length=200)
    price = models.DecimalField(max_digits=6, decimal_places=2)
    category = models.ForeignKey(Category, on_delete=models.CASCADE, related_name="menu_items")
    def __str__(self):
        return self.title
```

2. Model Relationships

One-to-Many (ForeignKey)

One object relates to multiple child objects (e.g., One Category has many MenuItems).

Many-to-Many

Objects on both sides relate to multiple instances using `ManyToManyField` (e.g., Orders and Items).

One-to-One

Direct 1:1 mapping via `OneToOneField` (e.g., User and UserProfile).

3. Database Migrations Workflow

Migrations propagate changes made to your models (adding a field, deleting a model) into the database schema.

Step & Command	Action Taken
<code>python manage.py makemigrations</code>	Inspects <code>models.py</code> and writes new migration scripts inside the <code>migrations/</code> directory.
<code>python manage.py showmigrations</code>	Lists all migrations in the project and indicates if they have been applied ([X]) or not.
<code>python manage.py migrate</code>	Executes pending migration files and creates/updates database tables.

4. Essential Django ORM Queries (QuerySet API)

```
# Fetch all records
items = MenuItem.objects.all()
# Filter records (WHERE clause)
cheap_items = MenuItem.objects.filter(price__lte=10.00)
# Get single object (Raises exception if not found)
item = MenuItem.objects.get(id=1)
# Create new record
MenuItem.objects.create(title="Tacos", price=8.50, category=cat_obj)
```

5. Django Forms API vs. ModelForms

Django forms automate input HTML generation, data validation, and security sanitization (CSRF protection).

Standard Form (forms.Form)

Constructed manually field-by-field. Ideal for non-database inputs like contact forms, search queries, or feedback forms.

ModelForm (forms.ModelForm)

Automatically generates form fields based directly on a Django Model's schema. Connects form submissions directly to database records.

6. Creating & Processing Forms

```
# forms.py from django import forms from .models import MenuItem class MenuItemForm(forms.ModelForm): class Meta: model = MenuItem fields = ['title', 'price', 'category']
```

```
# views.py (Handling GET and POST request lifecycle) from django.shortcuts import render, redirect from .forms import MenuItemForm def create_item(request): if request.method == 'POST': form = MenuItemForm(request.POST) if form.is_valid(): form.save() # Saves object directly to the database return redirect('menu_list') else: form = MenuItemForm() return render(request, 'item_form.html', {'form': form})
```

7. Form Validation & Sanitization

Validation Concept	Description & Implementation
<code>is_valid()</code> Method	Runs field-level validators, checks required fields, and populates the <code>cleaned_data</code> dictionary.
<code>cleaned_data</code>	A dictionary holding sanitized, python-typed values after successful validation.
Custom Field Validation	Add custom clean methods in the form class (e.g., <code>clean_price()</code>) to raise <code>ValidationError</code> .

8. Django Admin Interface

Django provides an automatic built-in admin portal for managing database records and administrative users.

```
# Register models in admin.py from django.contrib import admin from .models import MenuItem, Category @admin.register(MenuItem) class MenuItemAdmin(admin.ModelAdmin): list_display = ('title', 'price', 'category') # Columns displayed in table search_fields = ('title',) # Adds search bar list_filter = ('category',) # Adds filter sidebar
```

9. User Management, Groups & Permissions

Django contains a comprehensive authentication and authorization framework out-of-the-box.

Default Model Permissions

Django automatically generates 4 default permissions for every registered model:

- **Add:** `app.add_modelname`
- **Change:** `app.change_modelname`
- **Delete:** `app.delete_modelname`
- **View:** `app.view_modelname`

Users & Groups

Superuser: Has full system rights (`createsuperuser`).

Groups: Collect permissions to assign them to multiple users at once (e.g., "Managers", "Chefs").

10. Configuring External Databases (MySQL Setup)

By default, Django uses SQLite. For production applications, you can configure an external database like MySQL in `settings.py`.

```
# 1. Install driver: pip install mysqlclient # 2. Update settings.py DATABASES dictionary: DATABASES = { 'default': { 'ENGINE': 'django.db.backends.mysql', 'NAME': 'restaurant_db', 'USER': 'db_user', 'PASSWORD': 'secure_password', 'HOST': 'localhost', 'PORT': '3306', } }
```